
Laboratory Guide

Lesson 06 — BLE Gateway and Asynchronous Notifications

Communication Technologies for the Internet of Things

Academic year 2026–2027

Catarina Silva

ESTGA, University of Aveiro

Estimated duration: 2 hours

Contents

Context	2
Learning Outcomes	2
1 Preparation and Remote Access	3
2 Programmatic Discovery of Peripherals	3
3 Notifications and Forwarding	5
Questions for Discussion	5
References	6

Context

Lesson 05 built the BLE peripheral. This session builds the other end of the chain: the Raspberry Pi takes on the gateway role, is administered remotely, discovers the peripheral, subscribes to its notifications and forwards the readings to the IP network.

The Python programs are provided in the `codigo/` folder and are meant to be run and read, not written. The visualisation panel is built in Node-RED, without code.

Material

- Raspberry Pi with a prepared card, connected to the laboratory local network.
- Arduino with HM-11 shield and DHT11, assembled in Lesson 05.
- Personal computer, for SSH access.

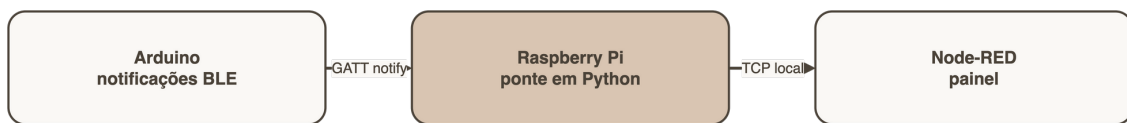


Figure 1: Architecture of the session

Learning Outcomes

- Access and configure remotely a node with neither monitor nor keyboard;
- Confirm that the BlueZ stack is operational on the gateway;
- Discover peripherals by programmatic scanning and interpret the RSSI obtained;
- Subscribe to notifications and observe the difference in traffic compared with periodic polling;
- Make the readings received available in a panel built without writing code.

1. Preparation and Remote Access

25 min

Objective

Establish a remote session with the Raspberry Pi and confirm that the Bluetooth adapter is operational.

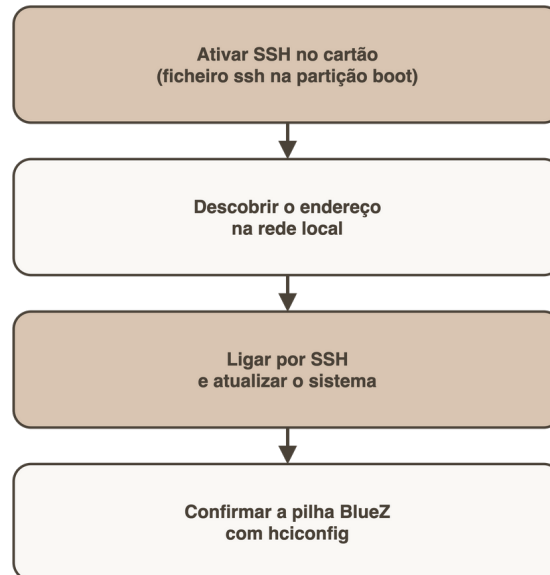


Figure 2: Gateway preparation sequence

```
ssh pi@<RASPBERRY_ADDRESS>
sudo apt update
hciconfig
sudo hciconfig hci0 up
```

1. Obtain the address assigned to the Raspberry Pi on the laboratory network.
2. Establish the SSH session and record the operating system version.
3. Confirm that the adapter appears in the listing and bring it to an operational state.

Why remote administration

In a real deployment, the gateway sits where the sensors are, with no peripherals attached. Working over SSH from the outset reproduces that condition and avoids the need for a monitor at each workstation.

2. Programmatic Discovery of Peripherals

30 min

Objective

Run the provided scanner and relate the result to the one obtained in Lesson 04 by command line.

```
sudo python3 codigo/varredor.py 15
```

1. Run the scanner and record the devices found, with their RSSI.
2. Move the Arduino closer to and further from the Raspberry Pi, repeating the scan, and record the variation in RSSI.
3. Compare the output with that of the `hcitool lescan` command from Lesson 04. What additional information is available now?

On RSSI

The value is given in dBm and is always negative; the closer to zero, the stronger the signal. It serves to rank devices by relative proximity, but is not a measure of distance: attenuation depends on obstacles, antenna orientation and reflections.

3. Notifications and Forwarding

45 min

Objective

Subscribe to the peripheral's notifications, forward them to the IP network and display them in a panel.

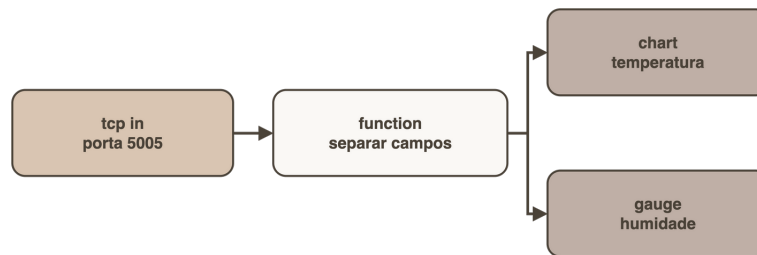


Figure 3: Node-RED flow to be built

```
sudo python3 codigo/ponte_ble_tcp.py <ARDUINO_ADDRESS>
```

The program connects to the peripheral, enables notifications by writing to the characteristic's configuration descriptor, and forwards each notification received to 127.0.0.1:5005.

Building the panel

1. Drag a **tcp in** node, in *Connect to* mode, pointing to 127.0.0.1:5005, with delivery of strings delimited by `\n`.
2. Drag a **function** node with two outputs and paste the provided content:

```
var v = msg.payload.split(",");
return [ { payload: parseFloat(v[0]) },
         { payload: parseFloat(v[1]) } ];
```

3. Connect the first output to a **ui-chart** node labelled *Temperature* and the second to a **ui-gauge** labelled *Relative humidity*.
4. Press **Deploy** and open the panel.

Observation to record

With the Arduino disconnected from power, the panel stops updating but the bridge does not terminate immediately. Record how long elapses until the connection is declared lost and relate that value to the supervision parameter studied in Lesson 04.

Further work

Replace the **tcp in** node with two nodes, one for each quantity, and organise the panel into two distinct groups. Separation into groups anticipates the organisation by topics that will be used in Lesson 12.

Questions for Discussion

1. Which functions does the gateway perform that the peripheral could not, and why?

2. If there were ten Arduinos instead of one, what changes would the bridge require?
3. Why would the local TCP connection used in this lesson not serve in a real deployment spread across several buildings?
4. What information about the devices in the room was it possible to collect without any prior authorisation?

References

- Gonçalves, P. *guia 1 — Remote admin*, *guia 3 — BLE Communication* and *aula9 — pythonNotif*, TCIoT, previous editions.
- bluepy library documentation: <https://ianharvey.github.io/bluepy-doc/>
- Raspberry Pi Foundation. Headless configuration and SSH access: <https://www.raspberrypi.com/documentation/>